All backend services are Dockerized and orchestrated via Kubernetes (Minikube), enabling features such as independent deployment of services, service discovery and internal routing via ClusterIP, autoscaling using Horizontal Pod Autoscaler (HPA) and environment-specific configuration through ConfigMaps and Secrets.

The platform uses a polyglot persistence strategy: MongoDB for semi-structured or dynamic data (Product, Chat, Wishlist) and PostgreSQL for transactional integrity (Order).

The frontend, built using Next.js, interacts with backend services over REST APIs and WebSocket connections, enabling real-time experiences such as live chat. Clerk.dev handles user authentication and access control using secure token-based sessions.

This architecture simulates real-world, production level deployments and demonstrates how open source technologies can power fully scalable and reactive applications.

Technology stack

This e-commerce platform is built using a suite of modern, open source technologies, each selected for its robustness, flexibility, and community support. The stack spans backend APIs, asynchronous messaging, containerisation, orchestration, and frontend development.

- ***Backend***

Node.js: A non-blocking, event-driven runtime perfect for handling high I/O workloads.

Express.js: A minimalist framework used to define RESTful API routes across services.

Kafka (Apache Kafka): Powers asynchronous, event-driven communication between services using topics like product-events, order-events, and chat-events.

kafkajs: Lightweight Kafka client for Node.js, used to implement producers and consumers.

- ***Databases***

MongoDB: Used in Product, Wishlist, and Chat services for handling semi-structured data with schema flexibility.

PostgreSQL: Used in the Order service where transactional integrity and relational data modelling are essential.

Prisma ORM: Used with PostgreSQL for schema validation and type-safe database operations.

- ***Frontend***

Next.js: React-based framework enabling server-side rendering, static site generation, and dynamic routing.

WebSocket: Used for real-time features such as chat and notifications.

Axios/Fetch API: For interacting with backend REST endpoints.

Clerk.dev: Provides secure user authentication, role-based access control, and JWT-based session handling.

- ***Infrastructure and deployment***

Docker: Each microservice is containerised to ensure consistent environments across development and deployment.

Kubernetes (Minikube): Orchestrates all services, managing pods, scaling, and service discovery.

Horizontal Pod Autoscaler (HPA): Automatically adjusts service replicas based on CPU utilisation.

This tech stack creates a solid foundation for building distributed, scalable applications while keeping development efficient and modular. Each tool plays a crucial role in transforming this project into a near-production-grade system.

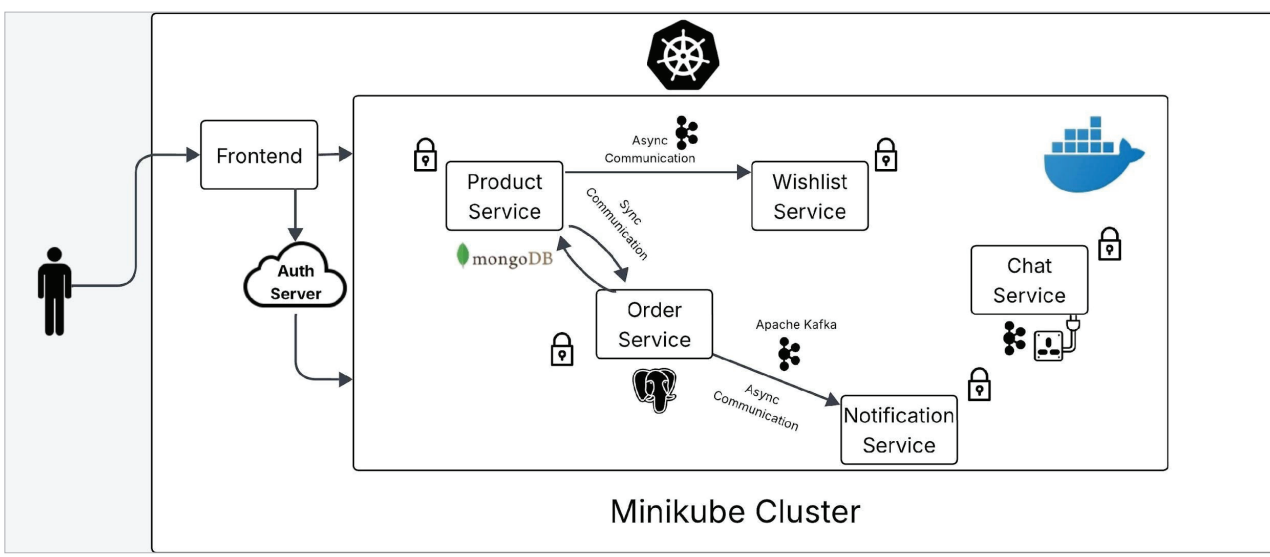

Figure 1: System architecture of the e-commerce platform